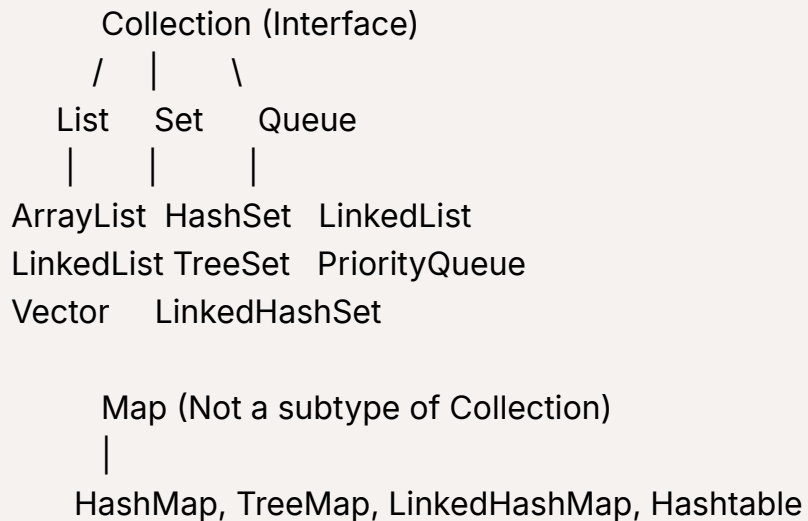


Java Collections

◆ COLLECTIONS HIERARCHY OVERVIEW



◆ 1. List Interface – `java.util.List`

✓ **Ordered**

✓ **Allows Duplicates**

📦 **Index-Based Access**

◆ Implementations:

- Best for: Random access, fast read

◆ `ArrayList` :

- Backed by dynamic array

◆ `ArrayList` – Random Access, Fast Read

Use Case:

- Storing items in a shopping cart (can add/remove, show in order, access by index)
- Maintaining user-selected filters in a UI (e.g., ["Red", "Large", "Cotton"])
- Maintaining history of visited URLs in a browser

Important Methods:

```
add(E e)
add(int index, E e)
get(int index)
set(int index, E element)
remove(int index)
size()
contains(Object o)
indexOf(Object o)
clear()
```

◆ **LinkedList** :

- Best for: Frequent insertions/deletions
- Doubly linked list

Extra Methods (in addition to **List**):

```
addFirst(E e)
addLast(E e)
removeFirst()
removeLast()
getFirst()
getLast()
```

◆ **LinkedList** – Frequent Insert/Delete

Use Case:

- Undo/Redo functionality in a text editor
- Task scheduling (insert/remove frequently)
- Playlist where songs can be added/removed from beginning or end

◆ **Vector** :

- Like `ArrayList`, but **synchronized** (thread-safe)

◆ **Vector** – Thread-Safe ArrayList

Use Case:

- Legacy code for multi-threaded applications where multiple threads access same list
 - Chat history in a messaging app (multi-threaded chat servers)
-

◆ **2. Set Interface** – `java.util.Set`

✗ **No Duplicates**

✗ Usually **Unordered** (except `LinkedHashSet`/`TreeSet`)

◆ **Implementations:**

◆ **HashSet** :

- Best for: Fast access, no duplicates
- Backed by **Hash Table**

◆ **HashSet** – Fast Lookups, No Order

Use Case:

- Maintaining a list of unique user IDs, emails, IPs
- Checking for duplicate entries during data import
- Storing visited URLs in a web crawler

```
add(E e)
remove(Object o)
contains(Object o)
isEmpty()
clear()
```

◆ **LinkedHashSet** :

- Preserves **insertion order**
- Slower than **HashSet**

◆ **LinkedHashSet** – Unique + Maintains Insertion Order

Use Case:

- Remembering order of form field validation errors (no duplicates, maintain order)
- Unique tags on blog posts (show in order entered)

◆ **TreeSet** :

- Stores elements in **sorted (natural or custom)** order
- Backed by a **Red-Black Tree**

◆ **TreeSet** – Sorted Set

Use Case:

- Auto-suggestions sorted alphabetically
- Maintaining leaderboard with scores
- Storing sorted set of available seat numbers in a theater

```
first()
last()
higher(E e)
lower(E e)
subSet(E from, E to)
```

◆ 3. Queue Interface – `java.util.Queue`

✓ FIFO (First In, First Out)

🚦 Used for **scheduling, message queues, task management**

◆ Implementations:

◆ `LinkedList` :

- Implements both `List` and `Queue`
- Doubly-linked structure

◆ `LinkedList` – General Queue/Deque

Use Case:

- Print queue (jobs handled in arrival order)
- Ticket reservation queue
- Background jobs to be executed by a scheduler

◆ `PriorityQueue` :

- Elements are ordered by **priority (natural or comparator)**

`PriorityQueue` – Based on Priority

Use Case:

- Task scheduler (high priority tasks executed first)
- Dijkstra's algorithm for shortest path in graphs
- CPU job scheduling in OS (e.g., Shortest Job First)

Important Methods:

```
offer(E e)    // Add element
poll()        // Retrieve and remove head
peek()        // Retrieve head without removing
element()     // Throws exception if empty
remove()      // Removes head
```

◆ 4. Deque Interface – `java.util.Deque`

✓ Double-ended queue

📦 Supports FIFO and LIFO

↻ Stack-like or queue-like behavior

◆ `ArrayDeque` :

- Faster than `Stack` or `LinkedList`

◆ `ArrayDeque` – Faster than Stack/LinkedList

Use Case:

- Browser history (back/forward stack)
- Palindrome checker (insert both ends, compare)
- Expression evaluation (infix to postfix using stack behavior)

Important Methods:

```
addFirst(E e)
addLast(E e)
removeFirst()
removeLast()
peekFirst()
peekLast()
```

◆ 5. Map Interface – `java.util.Map`

🔑 Stores data in **Key** → **Value** pairs

❌ No duplicate keys

✅ Allows duplicate values

◆ Implementations:

◆ `HashMap` :

- Fastest, unordered
- Allows one `null` key and many `null` values

◆ `HashMap` – Fastest, No Order

Use Case:

- Caching data (e.g., user sessions)
- Lookup of country code → country name
- Counting word frequencies in a document

Important Methods:

```
put(K key, V value)
```

```
get(Object key)
remove(Object key)
containsKey(Object key)
containsValue(Object value)
keySet()
values()
entrySet()
```

◆ **LinkedHashMap** :

- Maintains **insertion order**

◆ **LinkedHashMap** – Insertion Order Maintained

Use Case:

- LRU (Least Recently Used) Cache
- Config settings that should retain order
- Recently visited locations in a map app

◆ **TreeMap** :

- Sorted by **key**
- No **null** keys allowed

Use Case:

- Auto-complete suggestions sorted by string
- Leaderboard of users (score → user name)
- Time-based events (timestamp → event details)

◆ **Hashtable** (Legacy):

- Thread-safe version of HashMap
- ❌ No **null** keys/values

Hashtable – Legacy Thread-Safe Map

Use Case:

- Used in legacy enterprise applications
- Storing shared configuration settings across threads (older systems)

◆ 6. Utility Class – Collections

Helps manipulate collections (like sort, search, shuffle)

Key Methods:

```
Collections.sort(List<T> list)
Collections.reverse(List<?> list)
Collections.shuffle(List<?> list)
Collections.max(Collection<?> coll)
Collections.min(Collection<?> coll)
Collections.frequency(Collection<?> c, Object o)
Collections.binarySearch(List<? extends Comparable> list, T key)
Collections.synchronizedList(List<T> list)
```